

✓ Lab 4.1: Tiền xử lý, Phát hiện Biên, Hình học và Đặc trưng Góc

Phần 0: Chuẩn bị môi trường và Tải dữ liệu

Trước khi đi vào các thuật toán cụ thể của xử lý ảnh, sinh viên cần thiết lập không gian làm việc, import các thư viện cốt lõi và định nghĩa một số hàm phụ trợ giúp việc hiển thị ảnh dễ dàng hơn.

1. Kết nối Google Drive (Dành cho Google Colab) Để có thể đọc ảnh từ Drive cá nhân hoặc lưu kết quả thực hành, chúng ta cần "mount" Google Drive vào môi trường Colab.

```
from google.colab import drive
import os
from os.path import join

# Mount Google Drive
root = '/content/drive/'
drive.mount(root)

# Khởi tạo đường dẫn đến thư mục chứa bài thực hành của bạn
# Sinh viên có thể tùy chỉnh đường dẫn này theo cấu trúc thư mục cá nhân
lab_path = join(root, "My Drive/ComputerVision-IT23M/ThucHanh_Chuong1/data/")

# Tự động tạo thư mục nếu nó chưa tồn tại
os.makedirs(lab_path, exist_ok=True)
print(f"Thư mục làm việc hiện tại: {lab_path}")

Mounted at /content/drive/
Thư mục làm việc hiện tại: /content/drive/My Drive/ComputerVision-IT23M/ThucHanh_Ch
```

2. Import các thư viện cần thiết Các thư viện nền tảng cho thị giác máy tính và tính toán khoa học bao gồm OpenCV (`cv2`), Numpy (`numpy`), và Matplotlib (`matplotlib.pyplot`) để hiển thị ảnh.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# In ra phiên bản OpenCV đang sử dụng để kiểm tra
print("OpenCV Version:", cv2.__version__)

OpenCV Version: 4.13.0
```

3. Hàm phụ trợ hiển thị ảnh (Helper Function) Mặc định, hàm `cv2.imread()` của OpenCV sẽ đọc ảnh màu dưới định dạng **BGR** (Blue-Green-Red). Tuy nhiên, thư viện

Matplotlib lại hiển thị ảnh theo định dạng **RGB** (Red-Green-Blue). Nếu không chuyển đổi, ảnh hiển thị sẽ bị sai màu. Chúng ta sẽ viết một hàm phụ trợ nhỏ để xử lý vấn đề này cho toàn bộ bài lab.

```
def imshow_cv(title, img, figsize=(8, 6)):
    """
    Hàm hỗ trợ hiển thị ảnh OpenCV bằng Matplotlib.
    Tự động nhận diện ảnh màu hay ảnh xám để hiển thị cho đúng.
    """
    plt.figure(figsize=figsize)

    # Kiểm tra số chiều của ảnh (3 chiều = ảnh màu, 2 chiều = ảnh xám)
    if len(img.shape) == 3:
        # Chuyển đổi hệ màu BGR sang RGB
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(img_rgb)
    else:
        # Sử dụng colormap 'gray' cho ảnh xám
        plt.imshow(img, cmap='gray')

    plt.title(title)
    plt.axis('off') # Ẩn trục tọa độ
    plt.show()
```

4. Tải dữ liệu ảnh mẫu Ta sẽ sử dụng lệnh `!wget` để tải trực tiếp một vài ảnh mẫu từ internet về môi trường Colab, phục vụ cho các phần thực hành phát hiện biên và góc (ví dụ: ảnh tờ Sudoku và ảnh bàn cờ).

```
# Di chuyển vào thư mục làm việc
%cd "{lab_path}"

# Tải ảnh tờ báo Sudoku (Dùng cho phần Cắt ngưỡng và Canny/Hough)
!wget -q -O sudoku.jpg "http://aishack.in/static/img/tut/sudoku-original.jpg"

# Tải ảnh bàn cờ (Dùng cho phần phát hiện góc Harris/Shi-Tomasi)
!wget -q -O chessboard.jpg "https://raw.githubusercontent.com/opencv/opencv/master/

# Đọc thử ảnh và kiểm tra hàm hiển thị
img_sudoku = cv2.imread('sudoku.jpg')
img_chess = cv2.imread('chessboard.jpg')

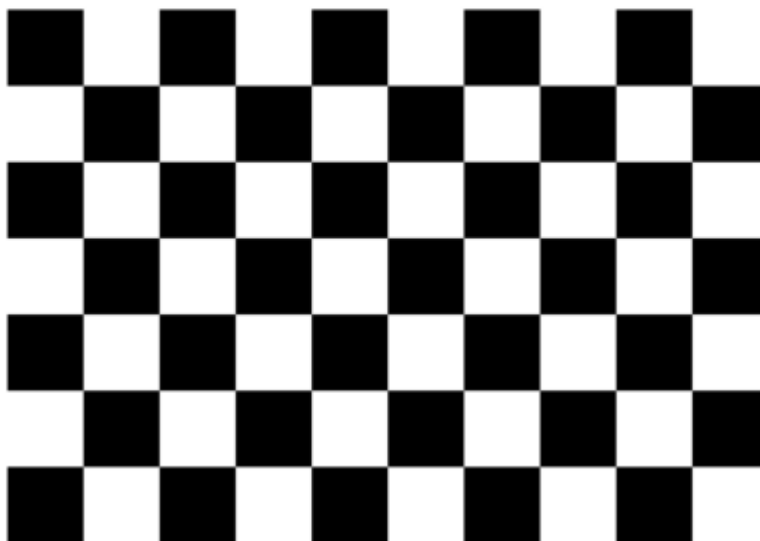
imshow_cv('Ảnh Sudoku Mẫu', img_sudoku, figsize=(6, 6))
imshow_cv('Ảnh Bàn cờ Mẫu', img_chess, figsize=(6, 6))
```

/content/drive/My Drive/ComputerVision-IT23M/ThucHanh_Chuong1/data

Ảnh Sudoku Mẫu



Ảnh Bàn cờ Mẫu



This is a file
OpenCV threshold
https://opencv.org/

- ✓ Phần 1: Kỹ thuật phân đoạn ảnh bằng cắt ngưỡng (Thresholding)

Mục tiêu: Hiểu và áp dụng phương pháp đơn giản nhất để tách vật thể khỏi nền. Kết quả thu được luôn là một ảnh nhị phân (chỉ gồm các pixel đen hoặc trắng) giúp làm nổi bật hình dáng tổng thể và giảm thiểu dung lượng lưu trữ.

1.1 Cắt ngưỡng toàn cục (Global Thresholding) và Otsu

Cắt ngưỡng toàn cục sử dụng duy nhất một giá trị ngưỡng (T) cho toàn bộ bức ảnh. Kỹ thuật này hoạt động hoàn hảo nếu ảnh có độ tương phản cao hoặc biểu đồ Histogram có dạng hai đỉnh (Bimodal Histogram). Tuy nhiên, nó có một "gót chân Achilles" cực lớn: thất bại hoàn toàn khi bức ảnh có chiếu sáng không đồng đều hoặc bị đổ bóng.

Để không phải "đoán mò" giá trị ngưỡng thủ công, ta kết hợp phương pháp **Otsu**, giúp máy tính tự động phân tích biểu đồ Histogram để tìm ra ngưỡng tối ưu nhất.

```
# Chuyển đổi ảnh Sudoku sang ảnh xám trước khi xử lý
gray_sudoku = cv2.cvtColor(img_sudoku, cv2.COLOR_BGR2GRAY)

# Áp dụng bộ lọc Gaussian để giảm nhiễu cục bộ trước khi cắt ngưỡng
blur_sudoku = cv2.GaussianBlur(gray_sudoku, (5, 5), 0)

# Cắt ngưỡng toàn cục thủ công (Giả sử chọn ngưỡng T = 127)
ret, thresh_global = cv2.threshold(blur_sudoku, 127, 255, cv2.THRESH_BINARY)

# Cắt ngưỡng tự động bằng thuật toán Otsu
ret_otsu, thresh_otsu = cv2.threshold(blur_sudoku, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f"Ngưỡng tối ưu do Otsu tự động tìm ra là: {ret_otsu}")

# Hiển thị kết quả (Bạn sẽ thấy vùng bị đổ bóng của tờ báo bị biến thành mảng đen)
imshow_cv('Cắt ngưỡng Toàn cục (Thủ công)', thresh_global)
imshow_cv('Cắt ngưỡng Otsu Tự động', thresh_otsu)
```


Ngưỡng tối ưu do Otsu tự động tìm ra là: 97.0

Cắt ngưỡng Toàn cục (Thủ công)

✓ 1.2 Cắt ngưỡng thích ứng (Adaptive Thresholding)

Để giải quyết bài toán ánh sáng phức tạp và bóng đổ, ta dùng cắt ngưỡng thích ứng. Thay vì dùng một ngưỡng chung, thuật toán sẽ tính toán ngưỡng riêng cho từng vùng lân cận nhỏ (ví dụ 11×11 pixel) xung quanh mỗi pixel, và cũng trừ đi một hằng số C để kiểm soát độ nhạy và loại bỏ nhiễu li ti ở vùng nền.

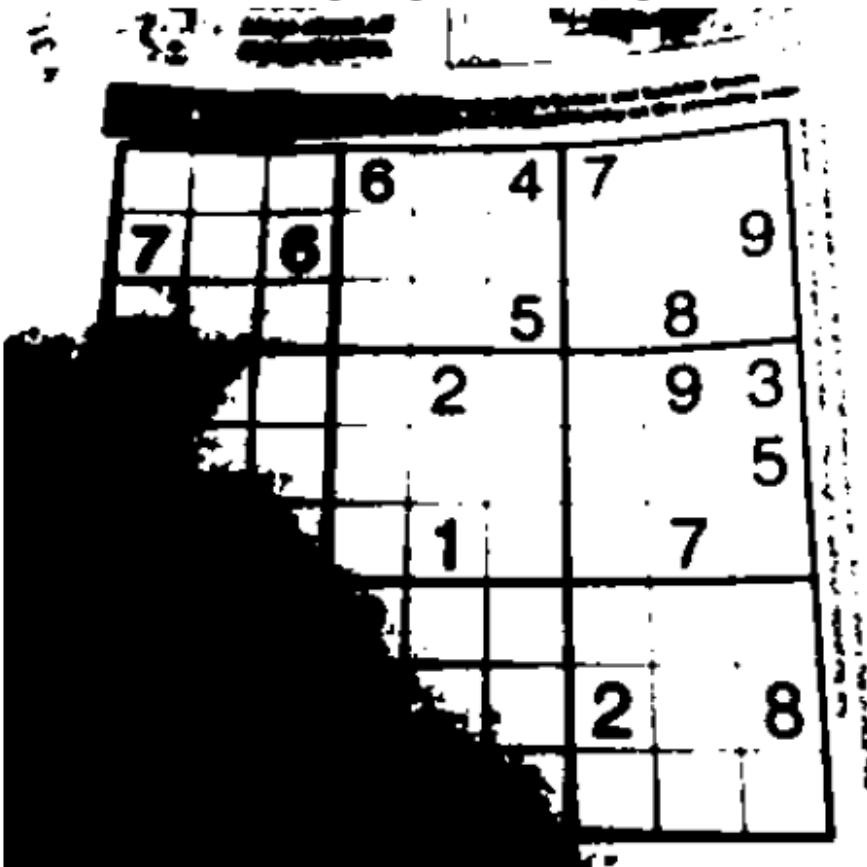
```
# Sử dụng phương pháp trung bình (MEAN) hoặc trung bình có trọng số (GAUSSIAN)
# Block Size = 11 (phải là số lẻ), Hằng số C = 2
thresh_adapt_mean = cv2.adaptiveThreshold(blur_sudoku, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                                         cv2.THRESH_BINARY, 11, 2)

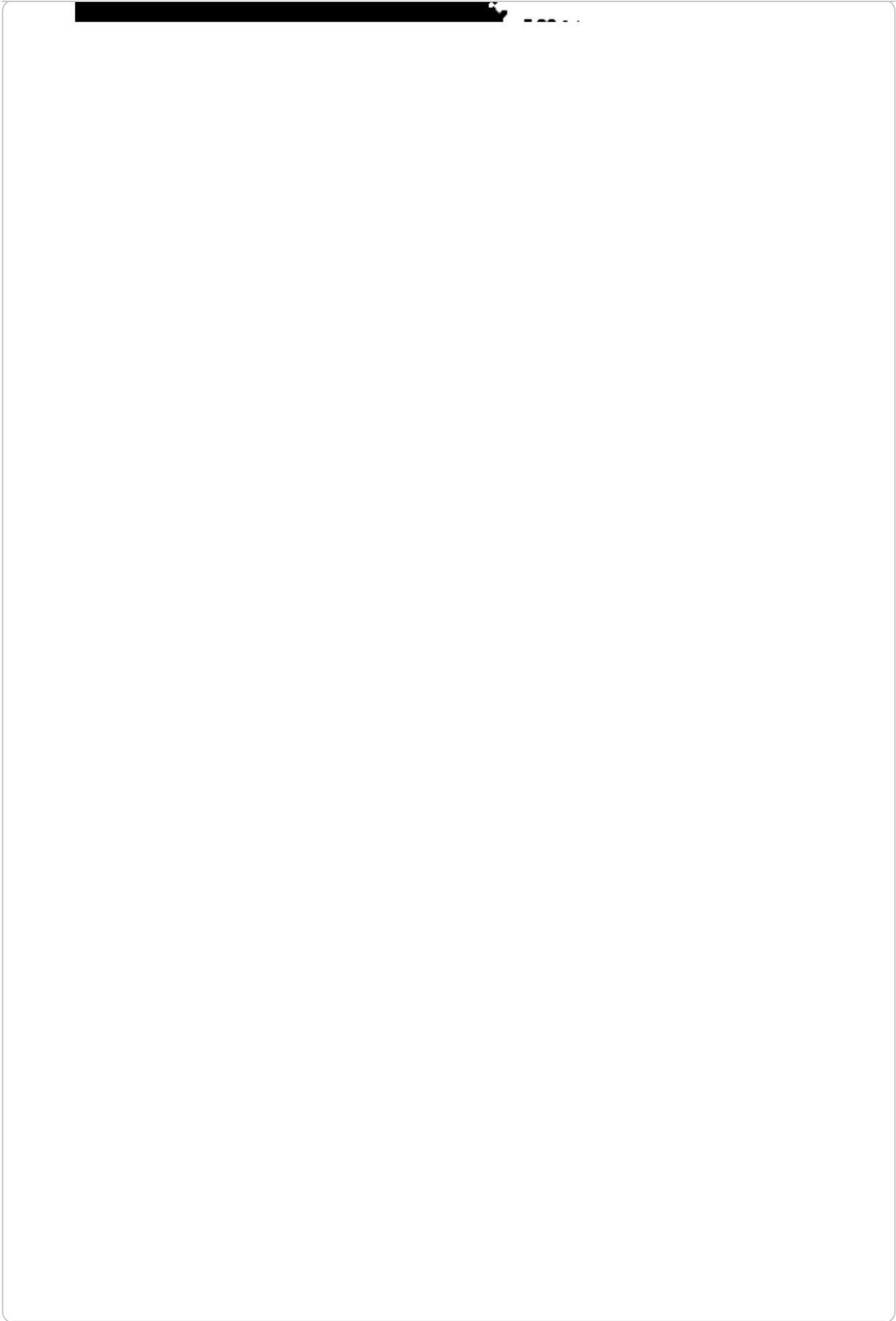
thresh_adapt_gauss = cv2.adaptiveThreshold(blur_sudoku, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                         cv2.THRESH_BINARY, 11, 2)

# Hiển thị kết quả để thấy lưới Sudoku được tách ra rõ nét bất chấp vùng bóng đổ
imshow_cv('Adaptive Thresholding (Mean)', thresh_adapt_mean)
imshow_cv('Adaptive Thresholding (Gaussian)', thresh_adapt_gauss)
```



Cắt ngưỡng Otsu Tự động





Adaptive Thresholding (Mean)

✓ Phần 2: Phát hiện biên (Edge Detection)

Mục tiêu: Tách xuất các đường biên, nơi cường độ sáng thay đổi đột ngột, giúp làm nổi bật cấu trúc hình học của đối tượng. Ta sẽ so sánh bộ lọc cơ bản Sobel và thuật toán tối ưu Canny.

2.1 Đạo hàm với bộ lọc Sobel

Sobel sử dụng phép đạo hàm bậc nhất để xấp xỉ Gradient theo hướng ngang (G_x) và dọc (G_y). Tuy nhiên, hạn chế chủ yếu của Sobel là tạo ra các đường biên rất "dày" (nhiều pixel) và cực kỳ nhạy cảm với nhiễu.

Lưu ý code: Khi tính đạo hàm, giá trị có thể mang dấu âm (chuyển từ sáng sang tối), nên ta phải dùng kiểu dữ liệu `cv2.CV_64F` để tránh mất dữ liệu biên.

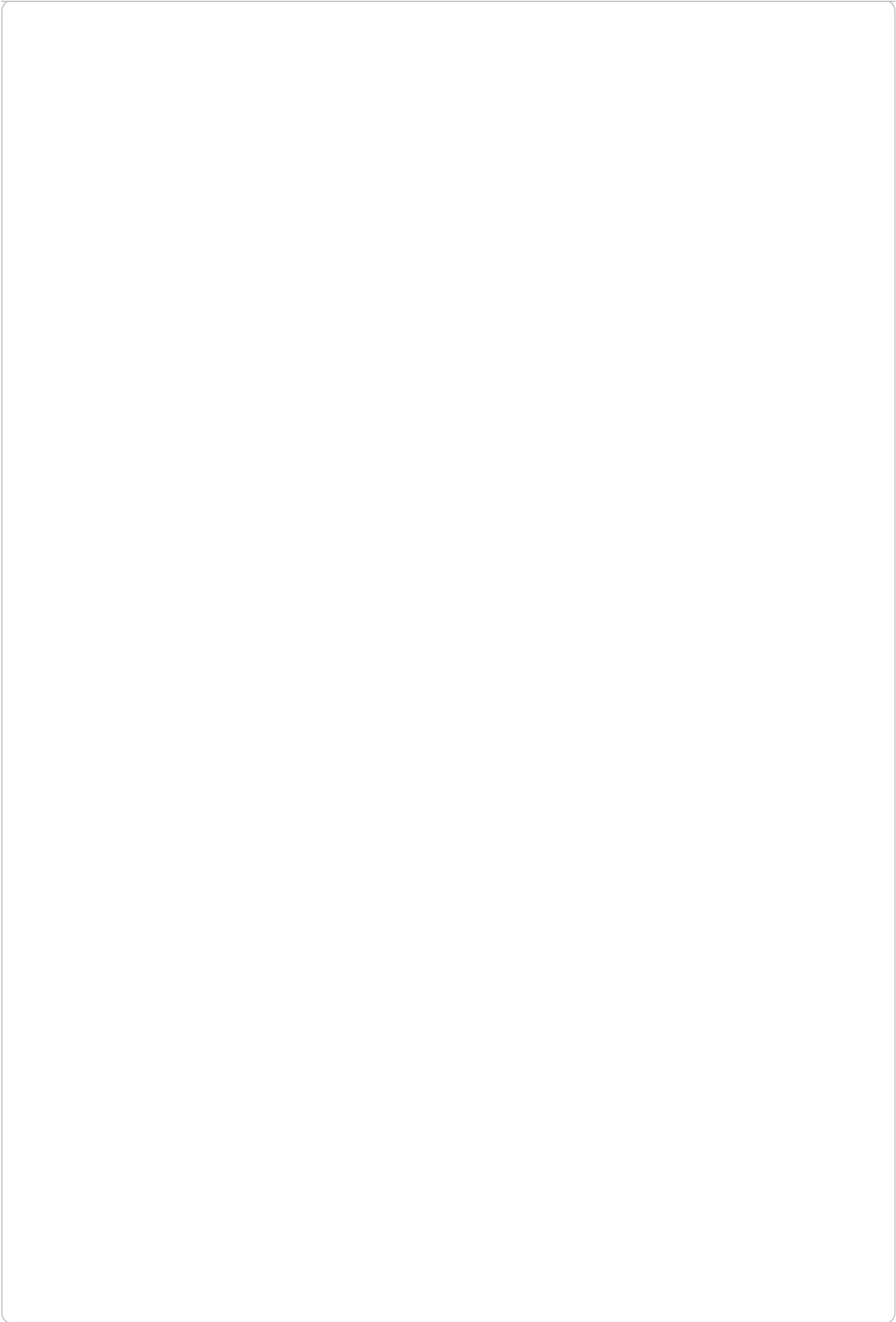
```
# Tính đạo hàm Sobel theo trục X và Y
sobel_x = cv2.Sobel(blur_sudoku, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(blur_sudoku, cv2.CV_64F, 0, 1, ksize=3)

# Lấy giá trị tuyệt đối và chuyển về lại kiểu uint8 (0-255)
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

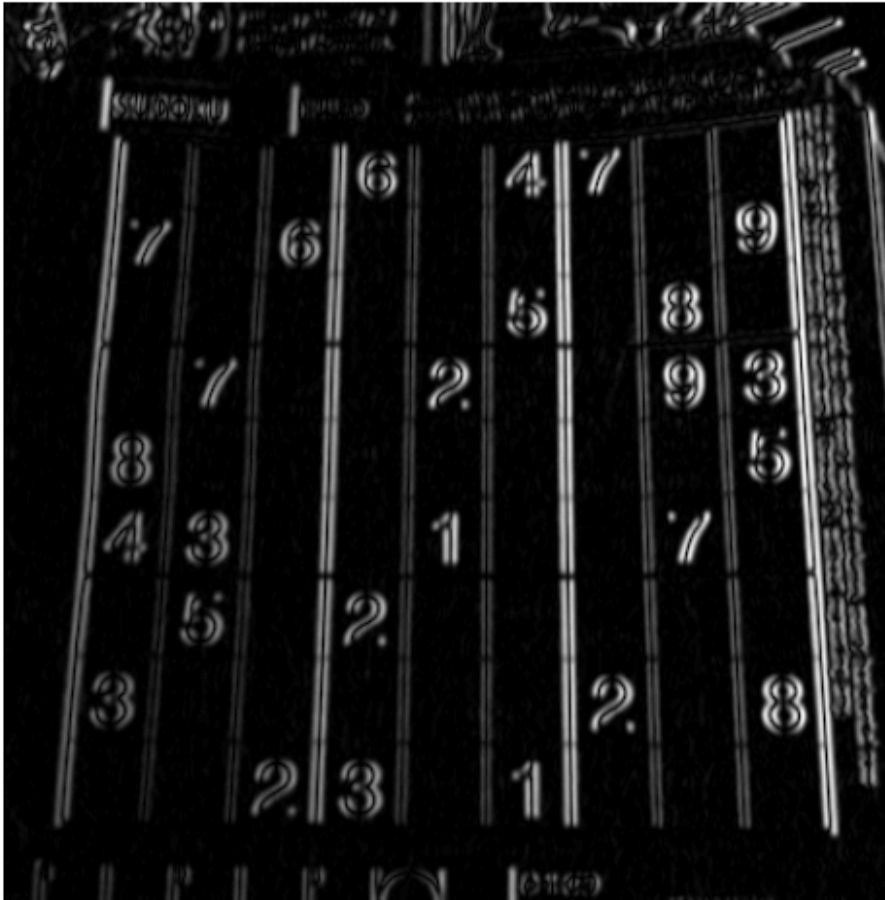
# Tổng hợp độ lớn Gradient của cả 2 hướng
sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

imshow_cv('Biên Sobel X', sobel_x)
imshow_cv('Biên Sobel Y', sobel_y)
imshow_cv('Biên Sobel Tổng hợp (Dày & Nhiều)', sobel_combined)
```





Biên Sobel X



Biên Sobel Y

2.2 "Tiêu chuẩn vàng" Canny

Thuật toán Canny vượt trội hơn hẳn các quy trình khác. Các bước: Tính gradient, Ước chế phi cực đại (làm mỏng biên xuống dung 1 pixel), và Phân ngưỡng kép (Hysteresis). Phân ngưỡng kép dùng ngưỡng cao (T_H) để bắt hết chính xác và ngưỡng thấp (T_L) để nối liền các nét bị đứt gãy.



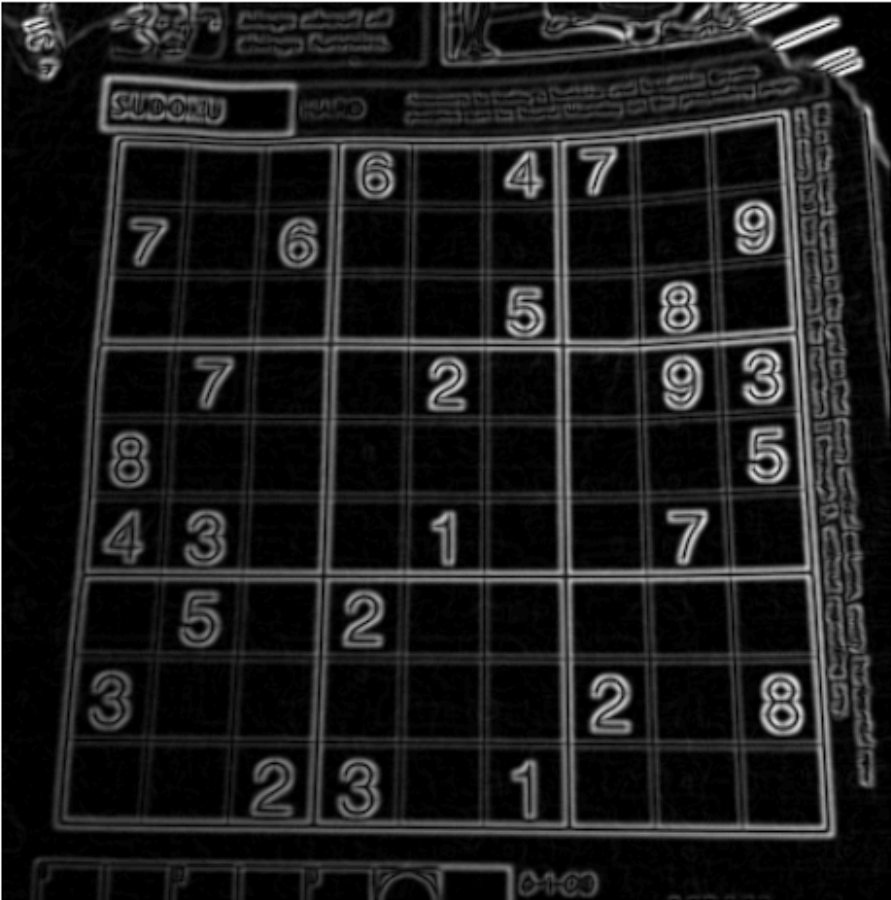
```
# Áp dụng Canny với Ngưỡng thấp (T_L) = 50 và Ngưỡng cao (T_H) = 150
edges_canny_1 = cv2.Canny(blur_sudoku, 50, 150)
```

```
# Thử nghiệm với Threshold lớn hơn để xem sự thay đổi (bỏ bớt biên yếu)
edges_canny_2 = cv2.Canny(blur_sudoku, 100, 200)
```

```
# Hiển thị để sinh viên so sánh độ "mảnh" và "sạch" so với Sobel
imshow_cv('Biên Canny (50, 150) - Liên tục & Mảnh 1 pixel', edges_canny_1)
imshow_cv('Biên Canny (100, 200) - Ít chi tiết hơn', edges_canny_2)
```



Biên Sobel Tổng hợp (Dày & Nhiều)



Biên Canny (50, 150) - Liên tục & Mảnh 1 pixel

Phần 3: Phát hiện hình học với Biến đổi Hough (Hough Transform)

Mục tiêu: Sinh ra một cách chuyển đổi từ không gian ảnh sang không gian tham số và cơ chế "bỏ phiếu" của thuật toán Hough để xây dựng một pipeline phát hiện làn đường cơ bản và nhận diện hình tròn.

3.1 Ứng dụng HoughLinesP: Xây dựng Pipeline phát hiện làn đường

Biến đổi Hough là một kỹ thuật để tìm kiếm các đường thẳng kéo dài vô tận qua toàn bộ bức ảnh. Để ứng dụng vào thực tế (như vạch kẻ đường) và tối ưu tốc độ, chúng ta sẽ sử dụng phiên bản cải tiến là **Biến đổi Hough xác suất (cv2.HoughLinesP)** giúp trả về chính xác tọa độ hai điểm đầu mút của đoạn thẳng.

Để phát hiện làn đường, chúng ta cần một quy trình (pipeline) kết hợp tạo Vùng quan tâm (ROI) nhằm loại bỏ nhiễu từ cây cối bầu trời.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Tải một ảnh đường giao thông mẫu để thực hành
!wget -q -O lane.jpg "https://raw.githubusercontent.com/udacity/CarND-LaneLines-P1/
img_lane = cv2.imread('lane.jpg')
lane_copy = img_lane.copy()

# Bước 1 & 2: Chuyển xám, làm mờ và tìm biên Canny
gray_lane = cv2.cvtColor(img_lane, cv2.COLOR_BGR2GRAY)
blur_lane = cv2.GaussianBlur(gray_lane, (5, 5), 0)
edges_lane = cv2.Canny(blur_lane, 50, 150)

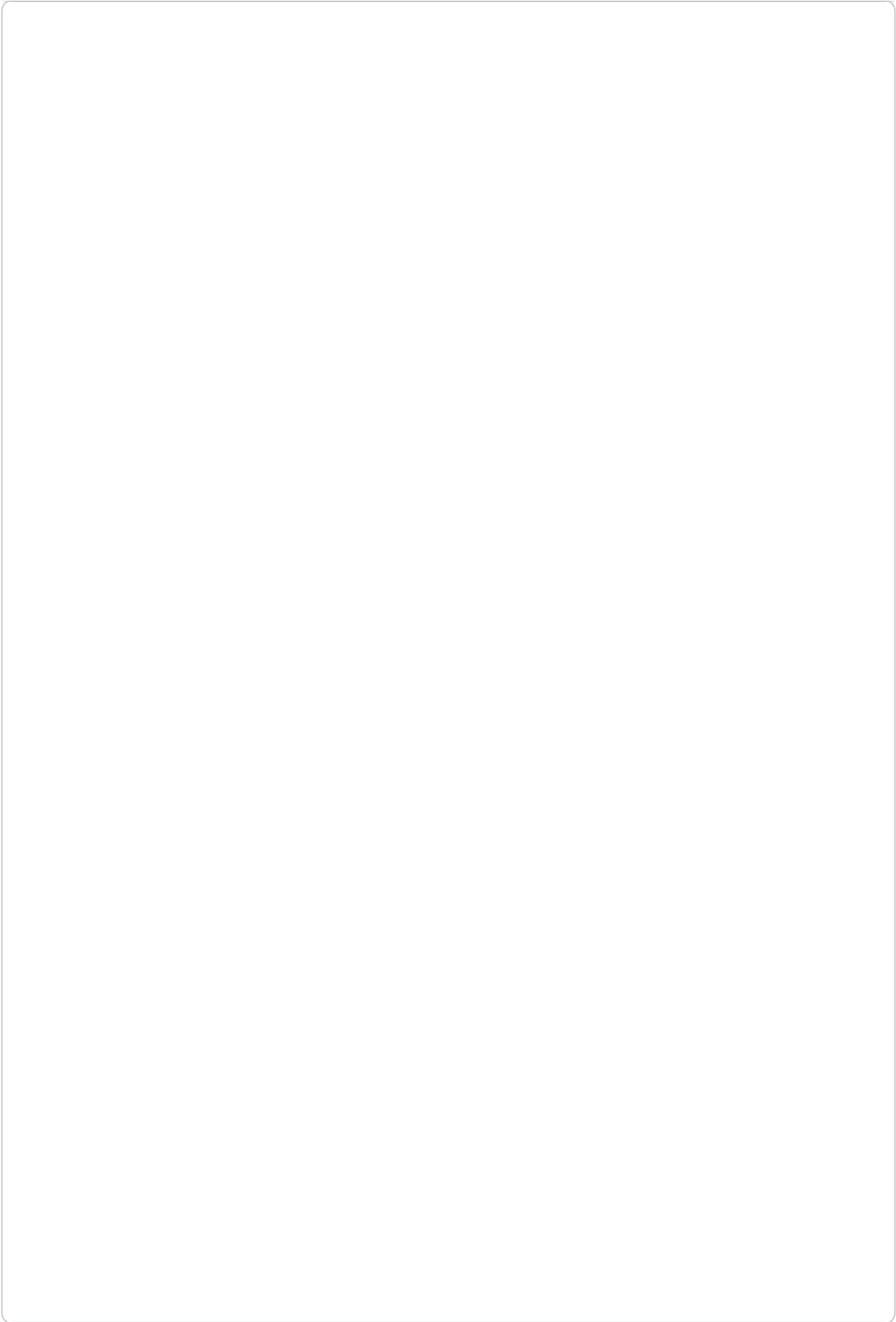
# Bước 3: Tạo Vùng quan tâm (ROI) hình thang bao phủ mặt đường
height, width = edges_lane.shape
mask = np.zeros_like(edges_lane)
# Định nghĩa các đỉnh của đa giác ROI (tùy chỉnh theo kích thước ảnh)
polygon = np.array([(100, height), (450, 320), (500, 320), (width, height)]), np.int_
cv2.fillPoly(mask, polygon, 255)

# Áp dụng bitwise_and để chỉ giữ lại các đường biên trong vùng ROI
masked_edges = cv2.bitwise_and(edges_lane, mask)

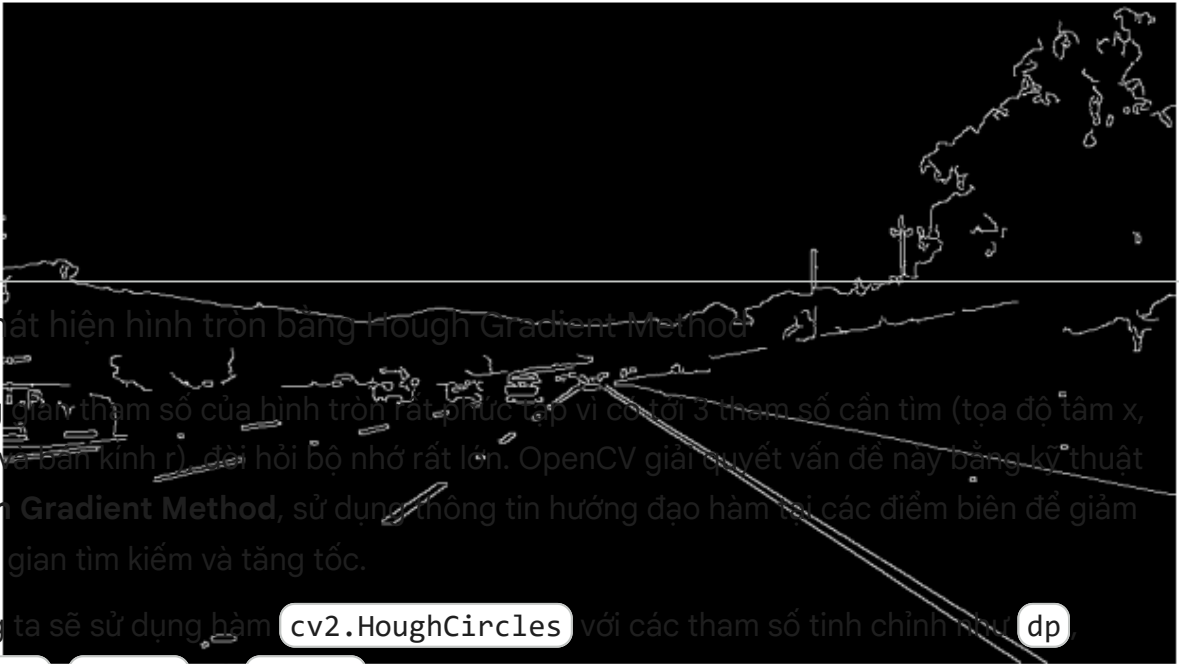
# Bước 4: Chạy HoughLinesP tìm các đoạn thẳng
# Thay đổi minLineLength và maxLineGap để nối các vạch đứt quãng
lines = cv2.HoughLinesP(masked_edges,
                        rho=1,
                        theta=np.pi/180,
                        threshold=50,
                        minLineLength=40,
                        maxLineGap=20)
```

```
# Bước 5: Vẽ các đường thẳng tìm được lên ảnh gốc
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0] # Corrected line: access line[0] to unpack
        cv2.line(lane_copy, (x1, y1), (x2, y2), (0, 255, 0), 5)

# Hiển thị các bước trong Pipeline
imshow_cv('Biên Canny toàn ảnh', edges_lane)
imshow_cv('Biên Canny trong vùng ROI', masked_edges)
imshow_cv('Phát hiện làn đường với HoughLinesP', lane_copy)
```



Biên Canny toàn ảnh



3.2 Phát hiện hình tròn bằng Hough Gradient Method

Không gian tham số của hình tròn rất phức tạp vì có tới 3 tham số cần tìm (tọa độ tâm x , tâm y và bán kính r), đòi hỏi bộ nhớ rất lớn. OpenCV giải quyết vấn đề này bằng kỹ thuật **Hough Gradient Method**, sử dụng thông tin hướng đạo hàm tại các điểm biên để giảm không gian tìm kiếm và tăng tốc.

Chúng ta sẽ sử dụng hàm `cv2.HoughCircles` với các tham số tinh chỉnh như `dp`, `minDist`, `param1`, và `param2`.

Biên Canny trong vùng ROI

```
# Tải một ảnh chứa các đồng xu
!wget -q -O coins.jpg "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/coins.jpg"
img_coins = cv2.imread('coins.jpg')

if img_coins is not None:
    img_coins_copy = img_coins.copy()

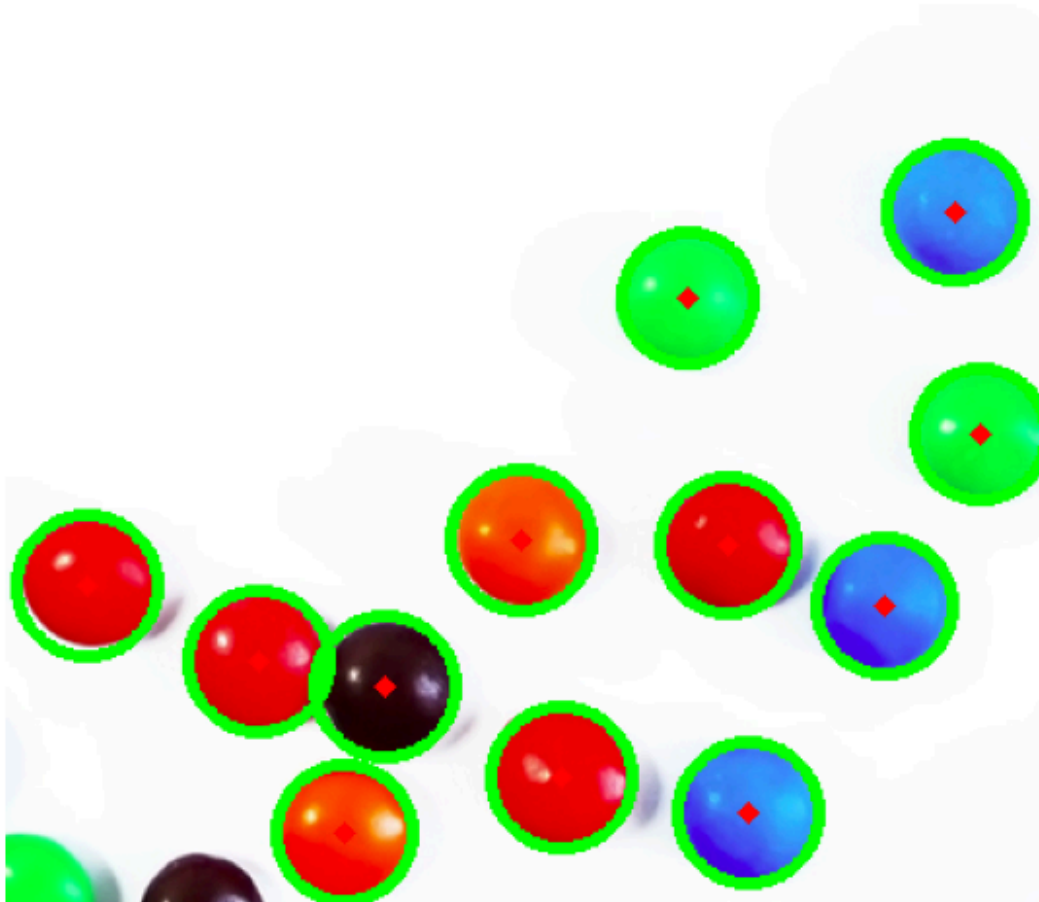
    # Tiền xử lý (Blur cực kỳ quan trọng đối với HoughCircles để tránh nhiễu)
    gray_coins = cv2.cvtColor(img_coins, cv2.COLOR_BGR2GRAY)
    blur_coins = cv2.medianBlur(gray_coins, 5)

    # Áp dụng HoughCircles
    circles = cv2.HoughCircles(blur_coins,
                               cv2.HOUGH_GRADIENT,
                               dp=1,           # Độ phân giải tích lũy so với ảnh
                               minDist=30,    # Khoảng cách tối thiểu giữa tâm 2 hình tròn
                               param1=50,     # Ngưỡng cao cho thuật toán Canny
                               param2=30,     # Ngưỡng bỏ phiếu, càng nhỏ càng r
                               minRadius=10,  # Bán kính tối thiểu
                               maxRadius=100) # Bán kính tối đa

    # Vẽ hình tròn và tâm lên ảnh
    if circles is not None:
        # Chuyển đổi tọa độ sang số nguyên
        circles = np.uint16(np.around(circles))
        for i in circles[0, :]:
            # Vẽ đường viền hình tròn (màu xanh lá)
            cv2.circle(img_coins_copy, (i[0], i[1]), i[2], (0, 255, 0), 3)
            # Vẽ tâm hình tròn (màu đỏ)
            cv2.circle(img_coins_copy, (i[0], i[1]), 2, (0, 0, 255), 3)
```

```
imshow_cv('Phát hiện hình tròn với HoughCircles', img_coins_copy)
else:
    print("Lỗi: Không thể tải được ảnh. Vui lòng kiểm tra lại đường dẫn hoặc kết r
```

Phát hiện hình tròn với HoughCircles



Gợi ý thảo luận cho sinh viên: Hãy thử thay đổi tham số `param2` xuống 10 hoặc lên 60 để xem thuật toán bắt đầu "tưởng tượng" ra hình tròn giả (false positives) hoặc bỏ sót các đồng xu như thế nào.

✓ Phần 4: Phát hiện Đặc trưng Góc (Corner Detection)

Mục tiêu: Hiểu được rằng góc là những điểm có tính định vị cao và ổn định nhất trên ảnh. Khảo sát hàm phản hồi góc dựa trên ma trận tự tương quan (Structure Tensor) và thực hành trích xuất góc bằng OpenCV.

4.1 Thuật toán Harris Corner Detector

Thuật toán Harris xác định góc dựa trên sự thay đổi cường độ sáng theo mọi hướng. Nó tính toán một hàm phản hồi R từ các trị riêng của ma trận cấu trúc. Nếu R có giá trị dương lớn, điểm đó được phân loại là góc.

Trong OpenCV, ta dùng hàm `cv2.cornerHarris()` với các tham số cốt lõi: kích thước vùng lân cận (`blockSize`), kích thước bộ lọc Sobel (`ksize`), và hằng số thực nghiệm k .

```
# Sử dụng ảnh bàn cờ (img_chess) đã tải ở Phần 0
chess_harris = img_chess.copy()
gray_chess = cv2.cvtColor(chess_harris, cv2.COLOR_BGR2GRAY)

# Chuyển đổi sang kiểu float32 theo yêu cầu của thuật toán Harris
gray_chess_float = np.float32(gray_chess)

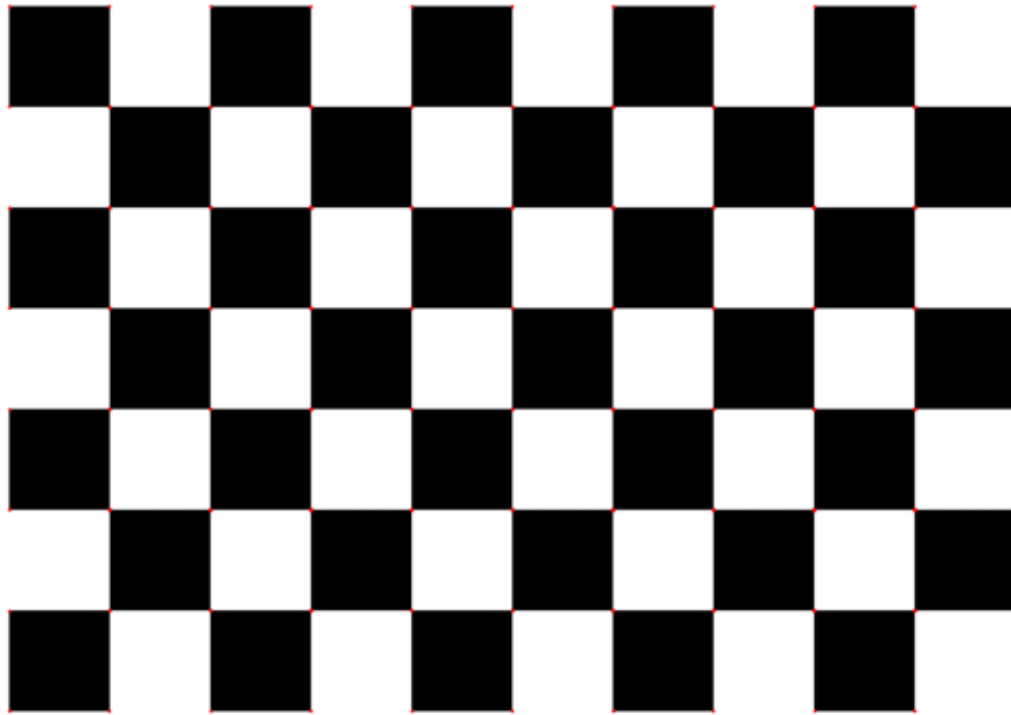
# Áp dụng thuật toán Harris
# blockSize = 2 (kích thước cửa sổ trượt 2x2)
# ksize = 3 (kích thước kernel Sobel)
# k = 0.04 (hằng số kinh nghiệm của Harris)
harris_response = cv2.cornerHarris(gray_chess_float, blockSize=2, ksize=3, k=0.04)

# (Tùy chọn) Giãn nở ảnh phản hồi để các điểm góc hiển thị to và rõ hơn
harris_response = cv2.dilate(harris_response, None)

# Phân ngưỡng để đánh dấu góc.
# Chỉ lấy các điểm có giá trị R > 1% so với giá trị R cực đại
threshold = 0.01 * harris_response.max()
chess_harris[harris_response > threshold] = [0, 0, 255]

imshow_cv('Phát hiện Góc bằng Harris', chess_harris)
```

Phát hiện Góc bằng Harris



This is a copy of the original image. It is not a new image.

✓ 4.2 Thuật toán Shi-Tomasi (Good Features to Track)

Năm 1994, Shi và Tomasi đã chứng minh rằng việc dùng trực tiếp giá trị nhỏ hơn trong hai trị riêng ($R = \min(\lambda_1, \lambda_2)$) sẽ cho kết quả các góc ổn định và phân bố đều hơn rất nhiều so với hàm phản hồi phức tạp của Harris.

OpenCV cung cấp sẵn hàm `cv2.goodFeaturesToTrack()`, tự động xử lý các khâu tính toán, sắp xếp và lọc để trả về đúng tọa độ (x, y) của các điểm góc tốt nhất. Ta sẽ dùng `cv2.circle()` để trực quan hóa các tọa độ này.

```
chess_shi_tomasi = img_chess.copy()

# Áp dụng thuật toán Shi-Tomasi
# maxCorners = 100: Tìm tối đa 100 góc tốt nhất
# qualityLevel = 0.01: Ngưỡng chất lượng (tối thiểu bằng 1% góc mạnh nhất)
# minDistance = 10: Khoảng cách tối thiểu giữa 2 góc liền kề (tránh tùm lại 1 chỗ)
corners = cv2.goodFeaturesToTrack(gray_chess, maxCorners=100, qualityLevel=0.01, r

# Chuyển mảng tọa độ về số nguyên để vẽ (Sửa lỗi np.int0 thành np.int32)
corners = np.int32(corners)

# Lặp qua từng góc và vẽ vòng tròn
for i in corners:
```

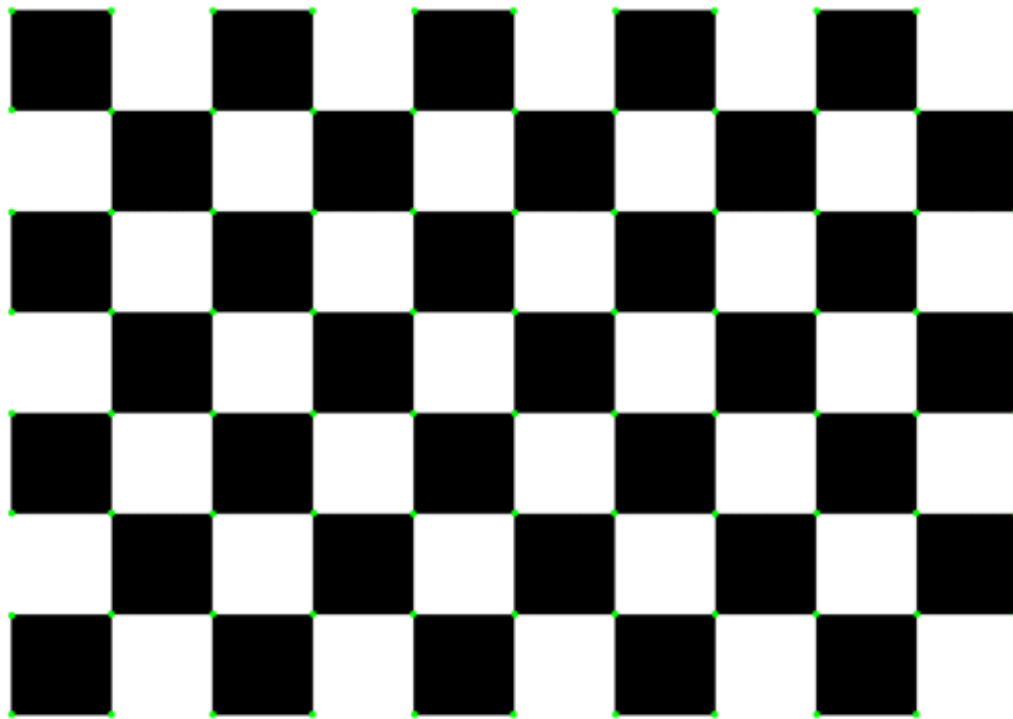
```

x, y = i.ravel()
# Vẽ vòng tròn tâm (x,y), bán kính 5, màu Xanh lá (Green), độ dày -1 (đồ đặc)
cv2.circle(chess_shi_tomasi, (x, y), 5, (0, 255, 0), -1)

imshow_cv('Phát hiện Góc bằng Shi-Tomasi', chess_shi_tomasi)

```

Phát hiện Góc bằng Shi-Tomasi



This is a 5x5
chessboard
https://opencv.org

Thảo luận cho sinh viên:

- Hỏi: Bạn thấy sự khác biệt nào về số lượng và sự phân bố của các điểm góc giữa Harris và Shi-Tomasi trên ảnh bàn cờ?
- Gợi ý: Thuật toán `goodFeaturesToTrack` (Shi-Tomasi) thường được ưu tiên dùng trong các bài toán *Theo dõi đối tượng (Tracking)* vì chất lượng góc trả về đồng đều hơn và dễ kiểm soát số lượng điểm thông qua tham số `maxCorners`. Tuy nhiên, cả hai đều có nhược điểm chung là **không bất biến với tỷ lệ** (scale invariance) - điều mà chúng ta sẽ giải quyết trong Lab 4.2 bằng thuật toán SIFT/SURF.

✓ BÀI TẬP

Chủ đề 1: Phân đoạn ảnh bằng kỹ thuật Cắt ngưỡng (Thresholding)

Bài tập 1: So sánh Cắt ngưỡng toàn cục (Global) và thuật toán Otsu